How CoBit's sampler works, and why score temperature breaks it

The maths of the continuous sampler in one notation, the four samplers we have (DDIM, EDM churn, the Euler–Maruyama SDE of Appendix D, ancestral) as members of one family, what each kind of temperature does to it, and what collapses the 500k CoBit-S results below τ = 0.7 · 13 Oct 2026 · CPU-only (no new GPU runs)

Two different failures, not one. Below τ=0.7 the 500k model fails differently with and without churn. **With churn** the readout is corrupted bit by bit: invalid token ids go 0% → 1.4% → 40% and the text becomes the right sentence with scrambled tokens. Cause: the entropic grid's last step (σ 0.157 → 0.002) is so large that a sharpened drift reflects the state through the prediction and amplifies the noise churn has just injected. **Deterministically** no token is ever invalid, but programs shrink (median 158 → 74 → 13 characters; 60% are just *return result* at τ=0.3): constant 1/τ on an ODE is strongly mode-seeking, and the dominant mode is ending early. **"CFG collapses later" is the σ schedule**: that curve used Karras, which has no large final step.

One rule covers every sampler. Each step moves the state towards the prediction D by a gain g; score temperature multiplies the gain by 1/τ, and a step is stable only while g/τ < 2 (and does not overshoot only while g/τ ≤ 1). EDM churn has g ≈ 0.29 on *every* step, whatever the step count; the ancestral step has g ≈ 2Δσ/σ ≈ 0.02. That is why ancestral sampling, or a smaller γ, can reach τ ≈ 0.1. DeepMind's logit temperature never overshoots at all: it sharpens D inside [0, 1] instead of extrapolating past it.

1 The process and the denoiser

CoBit diffuses bits $x_0 \in \{0,1\}^S$ (16 per token, 512 tokens) with a variance-exploding Gaussian process. The network predicts the per-bit posterior mean, a sigmoid of a learned logit plus an analytic matched-filter term, and Tweedie's formula turns it into the score:

$$x_\sigma = x_0 + \sigma\,\varepsilon,\ \varepsilon \sim N(0, I)$$
$$D(x, \sigma) = E[x_0 \mid x_\sigma = x] = \mathrm{sigmoid}(\, l_\theta(x, \sigma) + mf(x, \sigma)\,) \in (0,1)^S$$
$$s(x, \sigma) = \nabla \log p_\sigma(x) = (D - x) / \sigma^2$$

All samplers below integrate from $\sigma_{max} \approx 77$–80 down to $\sigma_{min} = 0.002$ on a grid $\sigma_0 > \ldots > \sigma_{N-1}$ and read out by thresholding D at σ_{min}. We write $\rho = \sigma_{i+1}/\sigma_i$ for the step ratio and $\Delta = \sigma_i - \sigma_{i+1}$.

2 One family of samplers

Georgios's entropy-gated SDE (paper, Appendix D) contains every sampler we use, indexed by the Langevin strength λ(σ) (r runs with decreasing σ):

$$dx = (1 + \lambda)\,\sigma\, s(x, \sigma)\, dr + \sqrt{2\lambda\sigma}\, dW_r$$

λ = 0 is the probability-flow ODE; λ = 1 is the standard reverse-time SDE (DDPM's); λ(σ) ∝ the entropy rate is CoBit's stochastic sampler. The four discretisations in the code are:

sampler (code)	one step	noise injected
DDIM / Euler, γ=0 (DDIMSampler)	$x' = D + \rho\,(x - D)$	none
EDM churn (DDIMSampler + γ)	$\hat\sigma = (1+\gamma)\sigma$, $\hat x = x + \sigma\sqrt{(1+\gamma)^2 - 1}\,\xi$, then $x' = \hat D + (\sigma'/\hat\sigma)(\hat x - \hat D)$	before the step, ∝ σ, every step
Euler–Maruyama (EulerMaruyamaSampler)	$x' = x - (1+\lambda)(\Delta/\sigma)(x - D) + \sqrt{2\lambda\sigma\Delta}\, z$	after the step, ∝ sqrt(Δ)

sampler (code)	one step	noise injected
ancestral, λ=1 (AncestralSampler, new)	x' = D + ρ² (x − D) + σ' sqrt(1 − ρ²) z	after the step, exact Gaussian bridge

All four are the same update written around the prediction: the residual to D is multiplied by (1 − g) and fresh noise is added. Only the **gain g** and the noise differ:

```
x' − D = (1 − g) (x − D) + noise
```

sampler	gain g per step	1024 Karras steps	entropic last step (σ 0.157 → 0.002)
DDIM	1 − ρ	median 0.0087, max 0.024	0.987
EDM churn γ	1 − ρ/(1+γ)	median 0.297 (γ=0.41)	0.991
Euler–Maruyama	(1+λ)(1 − ρ)	≈ churn at the same $\lambda_0 = \gamma(N-1)$	capped (em_step_gamma_cap)
ancestral	1 − ρ²	median 0.0173	1.000

The key contrast: EDM churn inflates σ by (1+γ) on *every* step, so its gain stays near γ/(1+γ) ≈ 0.29 however fine the grid is, whereas the ancestral and DDIM gains shrink with the step. Churn is a large step taken N times; the ancestral sampler takes N small ones.

3 What the three temperatures do to the step

Score temperature (Sigma; what we ran)

$D_\tau = x + (D - x)/\tau \Rightarrow x' - D_\tau = (1 - g)(x - D_\tau) \Leftrightarrow x' - D = (1 - g/\tau)(x - D)$

The gain becomes g/τ. The step is **stable only if g/τ < 2** (otherwise the residual grows geometrically and the state diverges) and **does not overshoot only if g/τ ≤ 1** (otherwise x jumps past D and the residual — including the noise just injected — is reflected and amplified by $|1 - g/\tau|$). This gives the observed limits: EDM churn on Karras needs $\tau > 0.31/2 \approx 0.15$; any sampler on the entropic grid's last step ($g \approx 0.99$) overshoots for $\tau < 0.99$.

Our earlier A1 temperature

$\kappa(\sigma) = (v + \sigma^2)/(\tau v + \sigma^2)$ multiplies the gain by $\kappa \in [1, 1/\tau]$, near 1 at high σ: the same effect, confined to the end of the trajectory.

Logit temperature (DeepMind, Simplex Diffusion Models, App. I.1 eq. 25)

$D_T = E_{c \sim \mathrm{softmax}(\zeta/T)}[c] \in [0,1]^{16}$ per token, $\zeta(c)$ = token logit of code c

The prediction is sharpened *inside* [0, 1] and then used in the unchanged step, so $x' = D_T + \rho(x - D_T)$ is a convex combination: the gain is unchanged and the step can never overshoot or diverge, at any T. This is why their T = 0.1 works with churn κ = 1, and why score temperature does not. CoBit already implements it (`--posterior_temp` with `--posterior_temp_space token`, target “full”); the rolling sampler now has it as `logit_temp`.

Why a deterministic sampler collapses even when it is stable

On a Gaussian toy the PF-ODE with drift ÷ τ ends with variance $v^{1/\tau}(v + \sigma_{max}^2)^{1-1/\tau}$ instead of the tempered τv: the whole trajectory contracts onto the dominant mode. Noise injection (churn, λ > 0) keeps re-spreading the state and counteracts this. In TinyGSM the dominant mode is the end of the program (most of the 512 positions are EOS/PAD).

4 What collapses the 500k model below τ = 0.7

sampler (500k EMA, entropic, 1024 steps)	τ	accuracy %	programs run %	invalid tokens %	median program length	body = 'return result' only
churn γ=0.41	1	28.1	63.5	0.00	166 chars	1%
churn γ=0.41	0.7	32.4	69.2	0.00	152 chars	1%
churn γ=0.41	0.5	2.7	5.0	1.39	150 chars	1%
churn γ=0.41	0.3	0.0	0.0	40.40	399 chars	0%
deterministic	1	12.2	31.5	0.00	158 chars	1%
deterministic	0.7	24.7	63.1	0.00	146 chars	1%
deterministic	0.5	4.4	47.2	0.00	74 chars	32%
deterministic	0.3	0.3	12.7	0.00	13 chars	60%

- **Churn arm — readout corruption.** Invalid ids appear at τ=0.5 and dominate at τ=0.3, while program length is unchanged: the decoder emits the right sentence with flipped bits. Mechanism: on the entropic grid the last step goes from $\hat{\sigma} = 1.41 \times 0.157 = 0.22$ (after churn) to 0.002 with $g \approx 0.99$, so at $\tau \leq 0.5$ it reflects the freshly churned residual through D with gain ≥ 1 and flips bits that were already decided.
- **Deterministic arm — mode collapse.** No invalid ids; programs shorten until most are *return result*. This is the contraction of Section 3, not a step-size problem; no change to the last step will fix it.
- **“With CFG it collapses much later.”** The CFG checkpoint and CoBit-M were run on the Karras grid, whose largest steps are tiny (DDIM max g = 0.024; churn $g \approx 0.31$, hence their wall at $\tau \approx 0.15$). The 500k arm used the entropic grid. The difference is the schedule, not guidance.

5 Fixes, and what a toy predicts for them

Toy: independent bits with the exact posterior (no network, no context), 1024 steps, the real grids. Entry = share of bits whose readout flips the decision held one step earlier (0 is ideal), or “diverges”. It isolates the integrator; it says nothing about mode collapse or model error.

sampler (toy)	grid	final step	τ=0.5	τ=0.3	τ=0.2	τ=0.1	τ=0.05
churn γ=0.41	entropic	sharpened	0.12%	8.1%	23%	diverges	diverges
churn γ=0.41	entropic	unsharpened	0	0	0	diverges	diverges
churn γ=0.41	karras	sharpened	0	0	0	diverges	diverges
churn γ=0.2	entropic	sharpened	0.012%	2.9%	11%	36%	diverges
churn γ=0.2	entropic	unsharpened	0	0	0	0	diverges
churn γ=0.2	karras	sharpened	0	0	0	0	diverges
churn γ=0.1	entropic	sharpened	0.0017%	1.2%	6.3%	22%	diverges
churn γ=0.1	entropic	unsharpened	0	0	0	0	diverges
churn γ=0.1	karras	sharpened	0	0	0	0	37%
ancestral	entropic	sharpened	0	0.047%	0.95%	7.3%	18%
ancestral	entropic	unsharpened	0	0	0	0	0
ancestral	karras	sharpened	0	0	0	0	0

- **Leaving the last step unsharpened** (new `--score_temp_final unsharpened`) removes every final-step flip, for every sampler and γ. It does not move the divergence wall.
- **Lower γ** moves the wall: on the Karras grid it follows $\tau > (1 - 1/(1+\gamma))/2$ (γ=0.41 → 0.15, γ=0.2 → 0.08, γ=0.1 → 0.045); the entropic grid has a few larger steps, so its wall sits a little higher (γ=0.1 still diverges at τ=0.05). Georgios's hunch is right on stability; the cost is less noise, i.e. more of the deterministic arm's mode collapse, so the best γ is an empirical question.
- **Ancestral** (new `--sampler_kind ancestral`) never diverges down to τ=0.05 and, on Karras or with the unsharpened last step, never flips. Its noise (σ' sqrt(1−ρ²) per step) is far below churn's, so whether it avoids mode collapse must be measured.
- **Logit temperature** avoids both integrator problems by construction (Section 3) and is the closest match to DeepMind's T = 0.1.

6 Proposed runs (nothing submitted)

All on the 500k EMA checkpoint, entropic grid, 1024 steps, 1319 problems, paired with the existing τ=1 cells; ~0.27 GH200-h per cell (measured), run in parallel (~20 min wall). Ordered so the cheap hypothesis test comes first; each later block is only worth running if the one before says so.

block	cells	question	GH200-h
A unsharpened last step, churn γ=0.41 and γ=0, τ ∈ {0.5, 0.3, 0.2}, with per-step diagnostics	6	Is the churn collapse the final step (predicted: invalid ids → 0, accuracy recovers)? Does the deterministic collapse persist (predicted: yes)?	1.6
B ancestral, unsharpened last step, τ ∈ {1, 0.7, 0.5, 0.3, 0.2, 0.1}	6	Can a low-noise exact sampler reach τ≈0.1 without mode collapse?	1.6
C γ ∈ {0.1, 0.2, 0.3} × τ ∈ {0.5, 0.3, 0.2, 0.1}, unsharpened last step	12	Does lower γ play better with temperature (stability vs mode collapse)?	3.2
D logit temperature (token space, full logit), churn γ=0.41, T ∈ {0.5, 0.3, 0.1}	3	DeepMind's operation on CoBit; cost of the codeword softmax to be checked on a 64-problem smoke first	≥ 0.8

Total ≈ 7–8 GH200-h. The rolling sampler needs no new mechanics: its η = 1 step is exactly the ancestral bridge above ($\text{ratio}^{(1+\eta^2)/2} = \rho^2$ at η = 1), and its score temperature already reads out the finalising step unsharpened; its own sweeps are the ones in TINYGSM.md.

Code (uncommitted): `DDIMSampler(score_temp_final=…)`, `AncestralSampler` in diffusion/continuous/samplers.py; eval flags `--score_temp_final`, `--sampler_kind ancestral`; score temperature now also reaches the EM sampler; tests in tests/test_sampler_fixes.py (6 new, 247 existing pass). Built by docs/pdf_reports/build_sampler_math.py.